

Agent-Based Simulation of Crowd Evacuation from the Cracovia Stadium

Piotr Andres¹

Paweł Podedworny²

¹Faculty of Computer Science
piotrandres@student.agh.edu.pl

²Faculty of Computer Science
podedworny@student.agh.edu.pl

2026

Abstract

This project presents an agent-based simulation of crowd evacuation from the Cracovia stadium. The study focuses on modeling pedestrian dynamics using Pygame, identifying infrastructural bottlenecks, and analyzing evacuation times under various emergency scenarios.

Keywords: agent-based modeling, crowd simulation, evacuation, stadium safety, Pygame

1 Introduction and review of related research

The simulation of crowd evacuation from a football stadium is a critical issue in the field of mass event safety. Due to the specific architecture of sports facilities, the process of leaving the stands often leads to the formation of congestion in so-called bottlenecks. Understanding and predicting crowd behavior in these tight spaces is essential for optimizing evacuation routes and ensuring the safety of attendees during emergencies [2]. According to the underlying principles of the Social Force Model proposed by Helbing and Molnár, pedestrian dynamics can be accurately described by a mixture of socio-psychological and physical forces. Individuals are motivated by an internal desire to reach their destination, while simultaneously being repelled by walls, obstacles, and other pedestrians in order to maintain a comfortable distance.

The object of this study is the Cracovia stadium located in Krakow. The stadium features a specific structural layout with designated VIP zones, restricted access from certain sides, and narrow vomitories (corridors leading from the stands to the external gates). When a crowd attempts to leave the stadium simultaneously, especially under panic conditions, phenomena such as the "fast-is-slow" effect can occur. In such situations, the physical arching of the crowd at the exits can significantly reduce the flow rate and increase the overall evacuation time [3]. As demonstrated by Parisi and Dorso, high desired velocities in narrow corridors inevitably lead to jamming and excessive pressure build-up, which is a critical factor that must be accounted for in stadium evacuations. Figure 1 illustrates the top-down architectural layout of the stadium, highlighting the key exit routes that will be subjected to simulation.

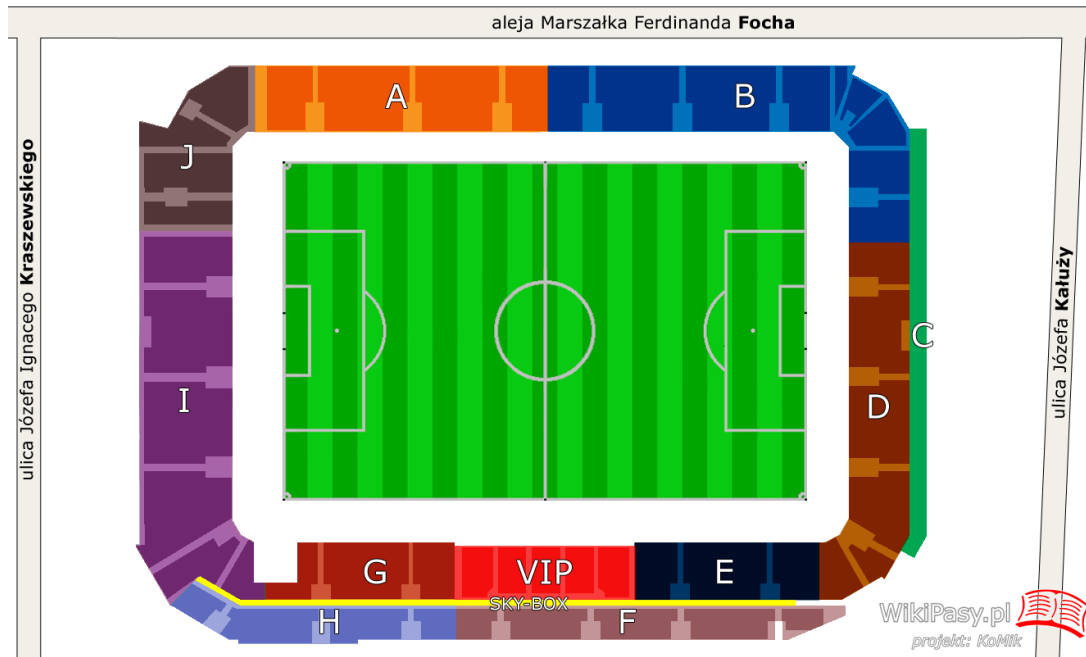


Figure 1: Top-down architectural plan of the Cracovia stadium, indicating major pedestrian flow routes.

Within the scope of literature and available solutions, two main approaches to pedestrian movement modeling have been identified and compared for the purpose of this project:

- **Cellular Automata (CA):** This method divides the simulation environment into a discrete grid where each cell is either empty or occupied by a single person. Movement is restricted to adjacent cells based on predefined transition probabilities. While computationally inexpensive and suitable for large-scale macroscopic flows, CA often lacks the fidelity required for complex, continuous human movement and struggles to accurately represent physical pushing or overlapping in highly dense crowds.
- **Agent-Based Modeling (ABM):** In this paradigm, each pedestrian is treated as an autonomous entity (agent) operating in a more continuous and granular space. Every agent maintains its own internal state, desired speed, and pathfinding logic [1]. This allows for a much more realistic representation of crowd dynamics, as agents can dynamically react to their local environment, alter their routes in real-time, and experience simulated physical forces.

Therefore, Agent-Based Modeling has been selected as the primary methodology for this project, as it provides the necessary flexibility to simulate complex behavioral patterns, such as panic, obstacle avoidance, and pushing at the stadium's vomitories.

The applicability of Agent-Based Modeling in the context of stadium evacuations has been widely demonstrated in recent literature. The first related study discusses modeling stadium evacuations within a smart city context using an agent-based system [5]. The initial motivation for the research was the 2019 Super Bowl hosted at the Mercedes-Benz Stadium in Atlanta; however, due to the complexity of the new facility, the authors opted to model the Georgia Tech Bobby Dodd Stadium. The primary goals of the project were to develop an Agent-Based Model (ABM) for a specific emergency scenario, generate various test cases to evaluate how smart city capabilities can assist in evacuations, and create an interface for visualizing the results. The authors chose the agent-based approach because it naturally replicates real-world events, provides

insights into the underlying causes of crowd phenomena, and establishes a robust framework for testing different safety strategies. The research primarily focused on safety protocols and evacuation efficiency. The simulated scenario involved a typical match day at the Bobby Dodd Stadium with a large crowd present, where a suspicious package is detected, prompting an immediate evacuation order. The study concentrated purely on the evacuation process rather than casualty modeling; the package does not detonate, acting solely as a hazard that agents attempt to avoid. Key variables analyzed in the study included the location of the package, the method by which attendees received the evacuation message, their familiarity with the escape routes, and instances of non-compliance with safety instructions. Furthermore, the article details the selection of NetLogo as the primary simulation tool, outlining how agents gather information from their surroundings to make reactive decisions within the established simulation framework. The results of the simulation demonstrated that utilizing smart city communication capabilities, such as real-time dynamic routing via mobile devices, significantly reduces the formation of bottlenecks and shortens the overall evacuation time. The authors concluded that the effectiveness of these systems heavily depends on the compliance rate of the crowd, highlighting that even partial adoption of smart routing instructions can efficiently balance the pedestrian load across all available stadium exits.

The second relevant study by Zhang, Wang, and Shi proposes a multi-agent model specifically designed to simulate the pedestrian evacuation process within a stadium environment [4]. In this research, the authors developed an individual decision-making framework where the movement direction of each agent is influenced by their distance to the exits, as well as the crowd density within their immediate field of view. A unique aspect of their model is the categorization of the crowd into heterogeneous demographic groups, such as young and elderly individuals of both genders, each possessing distinct physical attributes and decision-making weightings. Through various simulation scenarios, the study evaluated how the presence of architectural obstacles, the initial distribution of the crowd in the stands, and the positioning of exits impact the overall evacuation efficiency. Their findings demonstrate that incorporating demographic heterogeneity and local visual information into the agents' logic allows for a highly accurate reproduction of real-world stadium evacuations, providing a valuable framework for assessing and optimizing stadium architectural designs. This approach is highly relevant to the analysis of the Cracovia stadium, as it underscores the importance of evaluating specific layout configurations and potential structural bottlenecks on crowd throughput.

The main research goals of this project focus on practically applying these concepts to a real-world scenario. The initial phase involves mapping the topology of the Cracovia stadium based on available architectural plans into a navigable grid. Subsequently, an agent-based crowd movement model will be implemented using the Pygame library to simulate the evacuation process. Ultimately, this simulation will serve as a tool for identifying critical bottlenecks in the stadium's infrastructure and analyzing evacuation times under various scenarios, such as the blockage of specific exits or the presence of obstacles.

2 Problem formulation and proposed solution

2.1 System Architecture and Data Representation

The foundation of the simulation is built upon a data-driven architecture. The environment is entirely defined by a structured JSON configuration file. This file controls the dimensions of the application window, the physical properties of the crowd, and the structural layout of the stadium. Using a configuration file separates the logic of the simulation from its parameters. It allows researchers to quickly iterate and test multiple structural layouts without modifying the Python source code.

The spatial map of the stadium is represented as a two-dimensional array of string characters within the JSON structure. Each character corresponds to a specific structural element, such as a corridor, a set of stairs, a solid boundary, or an evacuation exit. The system translates these characters into functional objects using a dictionary of tile styles. This dictionary maps visual and physical properties to each tile. A crucial feature of this system is the decoupling of visual boundaries from physical collision boundaries. For example, a fence or a thin handrail might occupy an entire tile visually, but its physical collision area is strictly limited to a narrow rectangle. The algorithm achieves this by applying width and height ratios defined in the configuration, which is essential for accurately modeling the narrow constraints of stadium vomitories.

2.2 Global Navigation Algorithm

In a stadium evacuation, thousands of individuals need to find the optimal path to safety simultaneously. Calculating an independent route for every single agent at every frame would be computationally impossible. To solve this, the simulation implements a global navigation field generated by Dijkstra's algorithm.

During the initialization phase, the algorithm creates a grid matching the stadium layout. Every walkable tile is assigned an initial movement cost of infinity. The designated exit tiles are assigned a cost of zero and added to a priority queue structure. The algorithm then enters a loop. It repeatedly extracts the tile with the lowest cost from the queue and examines its valid neighbors. For each neighbor, the algorithm calculates a new movement cost. This new cost equals the current tile's cost plus the specific traversal weight of the neighboring tile.

The traversal weights are critical for realism. Moving across a standard floor tile adds a baseline cost of 1.0. However, moving onto a stairs tile adds a cost of 1.35. This mathematical penalty encourages agents to find flat paths if possible and accurately reflects the reduced walking speed on steep stadium grandstands. If the newly calculated cost for a neighbor is lower than its previously recorded cost, the grid is updated. The algorithm records the lower cost and also saves the coordinates of the tile from which the update originated. This process creates a continuous vector field pointing toward the exits. During the simulation, an agent only needs to query its current grid coordinates to instantly receive the optimal direction vector for its next step.

2.3 Physics Engine and Vector Integration

The movement of the agents is governed by a custom physics engine built with the Pygame library. The engine utilizes 2D vector mathematics to handle continuous motion. Each agent stores its position, velocity, and a target direction vector. At every frame of the simulation, the engine calculates the new state of the agent using Euler integration.

The integration process involves calculating the total force acting on the agent and multiplying it by the time elapsed since the last frame (delta time). This resulting velocity adjustment is added to the agent's current velocity. The final velocity vector is then multiplied by the delta time again to determine the exact positional change in pixels. The core of the movement logic lies in calculating the forces. The primary driving force is the desire to reach the exit. The algorithm checks the global navigation field to find the desired direction. It scales this direction by the agent's inherent maximum speed. The engine then calculates the difference between this desired velocity vector and the agent's actual velocity vector. This difference is multiplied by an acceleration factor, resulting in a force that constantly pulls the agent toward the exit route.

2.4 Social Repulsion Algorithm

To simulate the physical space taken up by humans, the system implements a repulsion algorithm inspired by the Social Force Model. Without this algorithm, agents would overlap and converge into a single point at the exits. At each frame, the system checks the distance between an agent and its neighbors.

The algorithm calculates a minimal acceptable distance, which is the sum of the radii of the two agents plus a configured personal space value. If the actual distance between two agents falls below this threshold, a repulsion vector is generated. The strength of this vector is strictly proportional to the depth of the overlap. The algorithm normalizes the distance vector and multiplies it by an overlap ratio. This ensures that a slight breach of personal space results in a gentle nudge, while a severe physical collision results in a massive repulsive force. This mathematical relationship accurately models the increasing pressure and pushing behavior observed in panic-driven stadium crowds.

A similar mathematical approach is applied to static obstacles. Agents perform spatial queries to detect nearby solid rectangles. If an agent approaches a wall within a defined sensory radius, a force pushes it away. The force is inversely proportional to the distance to the wall, preventing agents from scraping along fences and ensuring natural, curved movement around corners.

2.5 Collision Optimization via Spatial Partitioning

Checking the distance between every single agent in the simulation against every other agent creates an $O(N^2)$ algorithmic complexity. For an evacuation involving hundreds or thousands of people, this would reduce the frame rate to zero. The simulation solves this performance bottleneck by implementing a Spatial Partitioning technique.

At the beginning of each physics update, the algorithm distributes all active agents into a dictionary structure acting as a spatial hash map. The key for this map is a tuple representing the agent's current grid coordinates, and the value is a list of agents located in that cell. When calculating social repulsion, an agent no longer checks the entire crowd. The algorithm only queries the spatial hash map for the agent's current cell and the eight immediately adjacent cells. This optimization guarantees that an agent only performs distance calculations against individuals in its immediate physical vicinity. This drastically reduces the number of operations and allows the simulation to scale efficiently.

2.6 Iterative Hard Collision Resolution

While the calculated repulsion forces simulate psychological and light physical pressure, they are not always sufficient. High crowd density and numerical inaccuracies in the Euler integration can sometimes cause agents to visibly overlap or clip into walls. To guarantee strict physical boundaries, the simulation employs an iterative hard collision resolution strategy.

After all forces are applied and positions are updated, the engine runs a separate loop dedicated entirely to resolving critical overlaps. The number of passes is configurable; the experimental configuration uses one pass per physics update to maintain acceptable execution time for several thousand agents. The algorithm checks pairs of agents that are too close. It calculates the exact geometric vector required to separate them. The algorithm then instantly translates the positions of both agents along this vector, moving them apart by half of the overlap distance each.

A critical part of this resolution is modifying the velocities. Moving the agents apart solves the visual overlap, but their momentum would simply push them together again in the next frame. The algorithm calculates the separating speed using a vector dot product. It then applies a negative impulse to the velocities, effectively simulating an inelastic collision and dampening the kinetic energy of the crowd crush. A secondary function applies this exact same logic to walls, mathematically clamping the agent's position outside the bounding boxes of any solid tiles.

2.7 Agent Initialization and Diversity

The final algorithmic component ensures that the crowd behaves organically rather than robotically. If all agents had identical properties, the evacuation would look perfectly symmetrical. To prevent this, the initialization function introduces controlled randomness.

The algorithm first identifies all valid spawn tiles within the stadium. It then uses a mathematical distribution to evenly space the required number of agents across these valid areas. Once a position is chosen, the algorithm applies a random spatial jitter. This jitter offsets the agent slightly from the center of the grid cell, creating a natural, disorganized starting formation. Furthermore, the algorithm assigns a random speed multiplier to each individual. The multiplier ranges from 0.82 to 1.13. This creates a realistic mix of fast runners and slower walkers. Differences in speed create friction within the moving crowd. Faster agents bump into slower ones, causing localized congestion and turbulence, which perfectly models the "fast-is-slow" phenomenon critical to evacuation studies.

3 Experimental research results

3.1 Experimental Procedure

The final stadium model contains four evacuation passages on each of the northern and southern stands and three passages on each lateral stand. The corner areas were filled with triangular continuations of the neighbouring seating rows. The resulting geometry provides 11 262 unique initial agent positions. A regular application run uses 4 200 agents, whereas scenario comparison was performed with 1 000 agents to allow repeated experiments under identical conditions.

To remove rendering speed from the measurement process, a non-graphical execution mode was introduced. In this mode, the model is updated with a constant integration step of $1/30$ s

and runs without opening a window or rendering agents. Consequently, reported times are simulation times, not CPU execution times. Each run produces a semicolon-separated CSV time series containing the number of evacuated agents, current density, mean velocity, scenario-specific state variables, and final evacuation statistics. A sample invocation is:

```
1 python main.py --headless --scenario fire_north_sector \
2   --agents 1000 --seed 101 --max-duration 300
```

Listing 1: Headless execution of a fire scenario.

The comparison experiment used four conditions: an undisturbed baseline, a local fire hazard in the northern sector, a bombing incident in the eastern sector, and local panic on the western stand. The fire scenario starts after a seed-dependent time sampled from the interval 3–9 s. It begins as a small circular hazard and then grows across neighbouring stand cells, increasing their routing cost without removing walls, deleting seats, or closing exits. The bombing disables an eastern exit after 12 s and makes nearby people rapidly flee the incident location. The panic condition begins after 20 s and permanently changes affected agents by increasing speed, reducing desired personal distance, and adding random route decisions. Each condition was repeated for seeds 101, 202, and 303 with 1 000 initial agents. All runs completed; in the most difficult panic repetition (seed 202), clearance required 206.77 s.

3.2 Emergency Scenario Comparison

Table 1 reports values averaged over the three repetitions. Total clearance time is the time at which the last agent leaves the stadium. Mean and P95 evacuation time describe the experience of individual agents and therefore reveal deterioration even when the last evacuation time changes only slightly.

Table 1: Mean results for emergency scenarios with 1 000 agents and three random seeds.

Scenario	Clearance [s]	Mean [s]	P95 [s]	Max dens.	Panicked	Exposure [agent-s]
Baseline	98.53	31.50	56.87	3.33	0.00	0.00
Fire	98.53	31.51	56.87	3.33	0.00	18.01
Bombing	138.86	34.76	71.68	3.00	33.67	160.86
Panic	152.99	32.12	58.34	3.33	65.00	0.00

The baseline run completes in 98.53 s on average. After reducing the flame growth speed, the local fire hazard does not change the mean total clearance time in this experiment. Its effect is visible mainly through a small recorded exposure of 18.01 agent-seconds. This indicates that the hazard is local and slow enough for most agents to leave the affected area before it becomes a relevant bottleneck.

The bombing scenario has a stronger effect: total clearance is 40.33 s longer than the baseline and P95 rises by 14.81 s. The impulse away from the incident, persistent avoidance of the blast area, and removal of one exit temporarily disrupt ordered movement. Panic still causes the largest and most variable delay. Its average clearance time is 54.46 s above the baseline, even though the mean time for the majority of agents changes only moderately. This is caused by a long tail: random route choices and disturbed movement leave a small number of agents circulating or reaching an exit much later than the rest of the crowd.

Figure 3 shows the cumulative evacuation process for representative repetitions using seed 101. Until approximately 30 s the curves remain similar because agents are still moving through

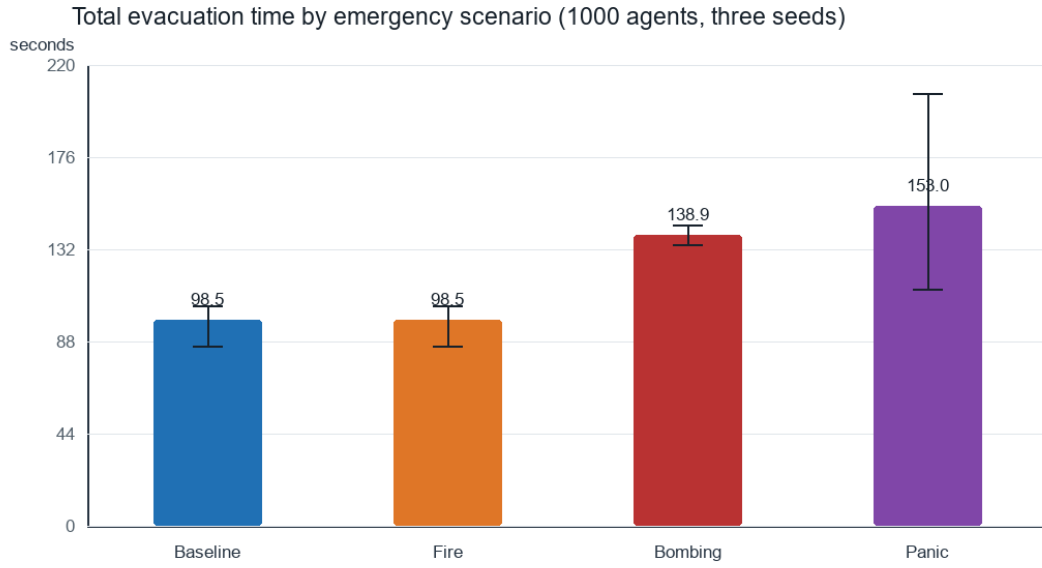


Figure 2: Average total evacuation time by scenario. Error bars show the minimum and maximum result across three seeds.

the seating rows. Differences arise when the flow reaches bottlenecks and when scenario effects modify access to exits or local agent behaviour.



Figure 3: Percentage of evacuated agents over simulated time for representative scenario runs.

3.3 Influence of Crowd Size

A second experiment examined the undisturbed scenario for six initial crowd sizes, using seed 101 and a maximum observation period of 300 s. The set includes the configured occupancy of 4 200 agents, an intermediate stress case of 7 731 agents, and the structural capacity of

11 262 unique starting places. No run required placing multiple agents in the same initial cell.

Table 2: Baseline evacuation results for different initial crowd sizes.

Agents	Status	Time [s]	Evacuated [%]	Mean [s]	Max dens.
500	completed	80.37	100.00	30.77	3
1000	completed	104.37	100.00	31.46	4
2000	completed	113.03	100.00	33.95	4
4200	completed	113.83	100.00	43.50	4
7731	limit reached	300.00	99.90	65.13	5
11262	limit reached	300.00	98.30	87.58	5

Increasing occupancy from 500 to 1 000 agents noticeably increases total evacuation time. Up to the regular occupancy of 4 200 agents, clearance time approaches approximately 114 s rather than growing linearly, because people are distributed over more seating cells while all fourteen passages operate in parallel. Nevertheless, the mean time experienced by an individual agent increases from 30.77 s to 43.50 s, confirming sustained congestion.

The additional high-occupancy runs change this conclusion for crowds approaching the geometric capacity. At the intermediate population of 7 731 agents, eight people remained in the stadium after the 300 s observation limit, and the mean recorded evacuation time increased to 65.13 s. At the maximum capacity of 11 262 initial positions, 191 agents remained after 300 s; only 98.30% had evacuated and the mean time among evacuated agents reached 87.58 s. Thus, the theoretical placement capacity cannot be interpreted as an operationally acceptable crowd size under the current exit configuration.

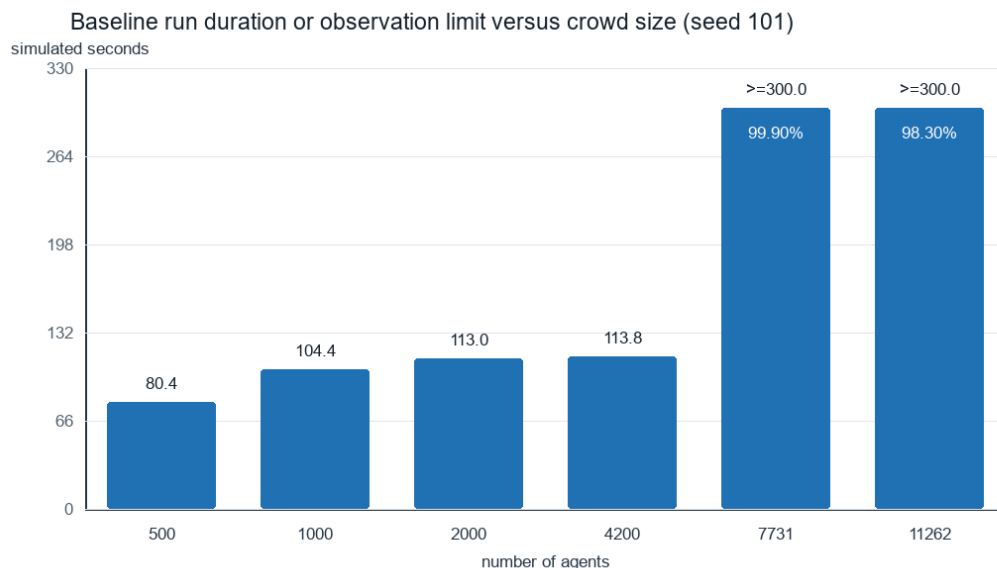


Figure 4: Clearance time or observation limit in the baseline scenario for increasing crowd sizes. The two largest runs did not clear fully within 300 s.

4 Summary and conclusions

4.1 Conclusions

The implemented agent-based model provides a configurable representation of the stadium stands, stairs, vomitories, exits, and corner seating areas. The use of a global navigation field combined with local repulsion and collision handling enables thousands of agents to evacuate through the same architectural bottlenecks. The headless mode makes the model suitable for reproducible experiments: the numerical simulation uses a fixed time step, while CSV output and automatically generated figures support subsequent comparison.

The conducted experiments demonstrate that behavioural disturbances can be more harmful than a local passable hazard. In the tested layout, the slowly growing fire area is substantially mitigated by route adaptation and by keeping all evacuation passages available. The bombing condition creates a stronger routing disruption due to immediate fleeing, continued avoidance of the incident area, and a closed passage. Local panic is the most adverse scenario: chaotic route decisions substantially lengthen the tail of the evacuation, with one repetition requiring more than twice the baseline mean clearance time. This confirms that evacuation safety cannot be evaluated solely through the nominal number of exits; the stability of crowd behaviour and route guidance is equally important.

The occupancy experiment indicates that the current architectural configuration supports the regular modelled audience of 4 200 agents without duplicated initial positions, clearing it in 113.83 s in the baseline run. However, crowds nearer the geometric capacity no longer clear within the 300 s observation period: 7 731 agents leave eight persons behind, while at 11 262 positions as many as 191 remain. The longer mean waiting and travelling time reveals congestion that would be masked if only the number of available initial places were considered. For this reason, future assessments should use clearance time, evacuation percentage, and distribution-based measures such as P95 jointly.

4.2 Limitations and Future Work

The current model intentionally focuses on movement and evacuation flow rather than injuries, smoke propagation, or physical pressure thresholds. Agents differ in speed but do not yet represent age, mobility restrictions, group behaviour, or compliance with instructions. Emergency scenarios are evaluated independently; a combined scenario, for example fire accompanied by panic, would likely produce more severe congestion. Moreover, the performance of dense simulations is still constrained by pairwise local collision calculations.

Further development should include heterogeneous pedestrian groups, dynamic signage or route announcements, combined hazards, and automated batch execution over broader ranges of seeds and occupancy. From the computational perspective, the next optimization target is the calculation of interpersonal forces and collision resolution. These additions would allow the simulation to support more detailed safety recommendations for the stadium layout and evacuation management procedures.

References

- [1] E. Bonabeau. “Agent-based modeling: Methods and techniques for simulating human systems”. In: *Proceedings of the National Academy of Sciences* 99.suppl_3 (2002), pp. 7280–7287.

- [2] D. Helbing and P. Molnár. “Social force model for pedestrian dynamics”. In: *Physical review E* 51.5 (1995), p. 4282.
- [3] D. R. Parisi and C. O. Dorso. “Microscopic dynamics of pedestrian evacuation”. In: *Physica A: Statistical Mechanics and its Applications* 392.18 (2013), pp. 3946–3954.
- [4] L. Zhang, J. Wang, and Q. Shi. “Multi-agent based modeling and simulating for evacuation process in stadium”. In: *Journal of Systems Science and Complexity* 27.3 (2014), pp. 430–444.
- [5] W. Zhang, V. Terrier, X. Fei, A. Markov, S. J. Duncan, M. Balchanos, and W. Sung. “Agent-based Modeling of a Stadium Evacuation in a Smart City”. In: *Winter Simulation Conference*. 2018.